Hospital Order Management System

CSCI-P532 · Spring 2026 · Project 2 — Pranav T P

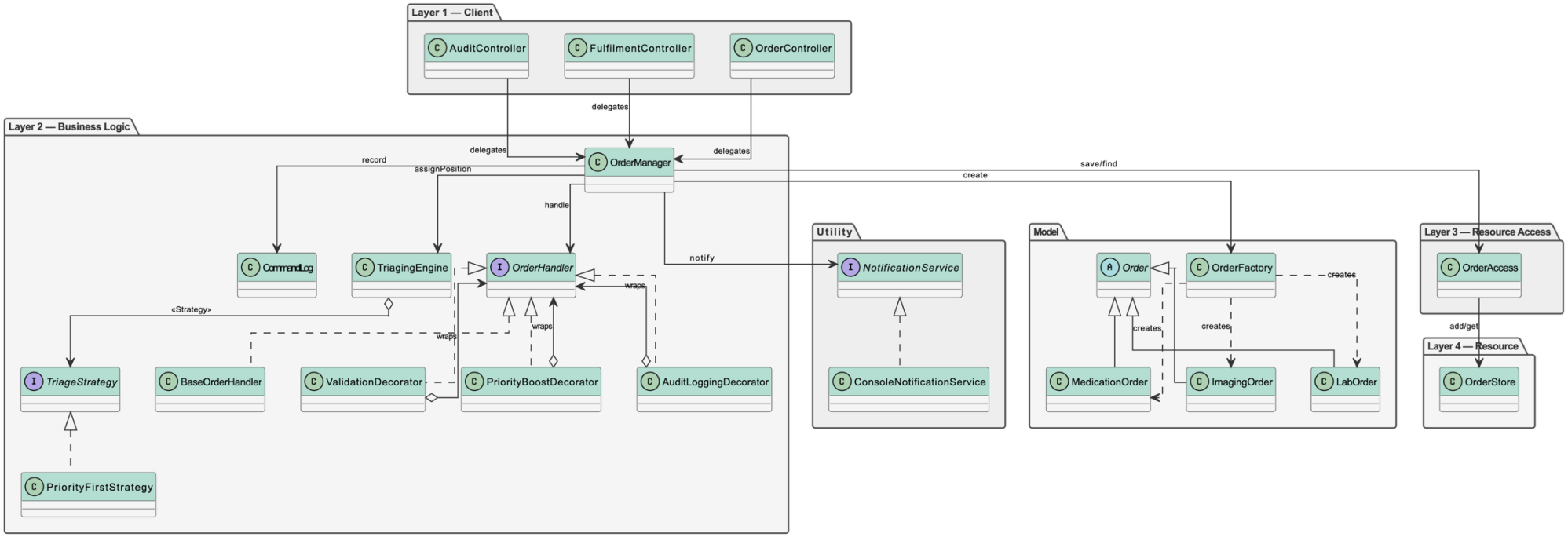
1. Four-Layer Architecture — Component Overview

Layer	Components	Responsibility
Layer 1 Client	OrderController FulfilmentController AuditController	Receives HTTP requests. Creates Command objects and delegates immediately to OrderManager. Contains zero business logic.
Layer 2 Business Logic	OrderManager (Manager) TriagingEngine (Engine) TriageStrategy (interface) PriorityFirstStrategy OrderHandler (interface) BaseOrderHandler ValidationDecorator PriorityBoostDecorator AuditLoggingDecorator Command + CommandLog	All decisions made here. OrderManager orchestrates all use cases. TriagingEngine runs the triage algorithm via TriageStrategy. Decorator chain processes every submitted order. Command objects wrap every action and are recorded in CommandLog.
Layer 3 Resource Access	OrderAccess	Only class allowed to talk to the order store. Exposes business verbs: saveOrder(), findOrderById(), listPendingOrders(), listAllOrders(). Hides how orders are stored.
Layer 4 Resource	OrderStore	Raw in-memory List. No business logic. No Spring annotations. Just holds data.
Utility (cross-cutting)	NotificationService (interface) ConsoleNotificationService	Called by any layer. Notifies relevant parties when order status changes. Console logging in Week 1; extensible via Decorator in Week 2.

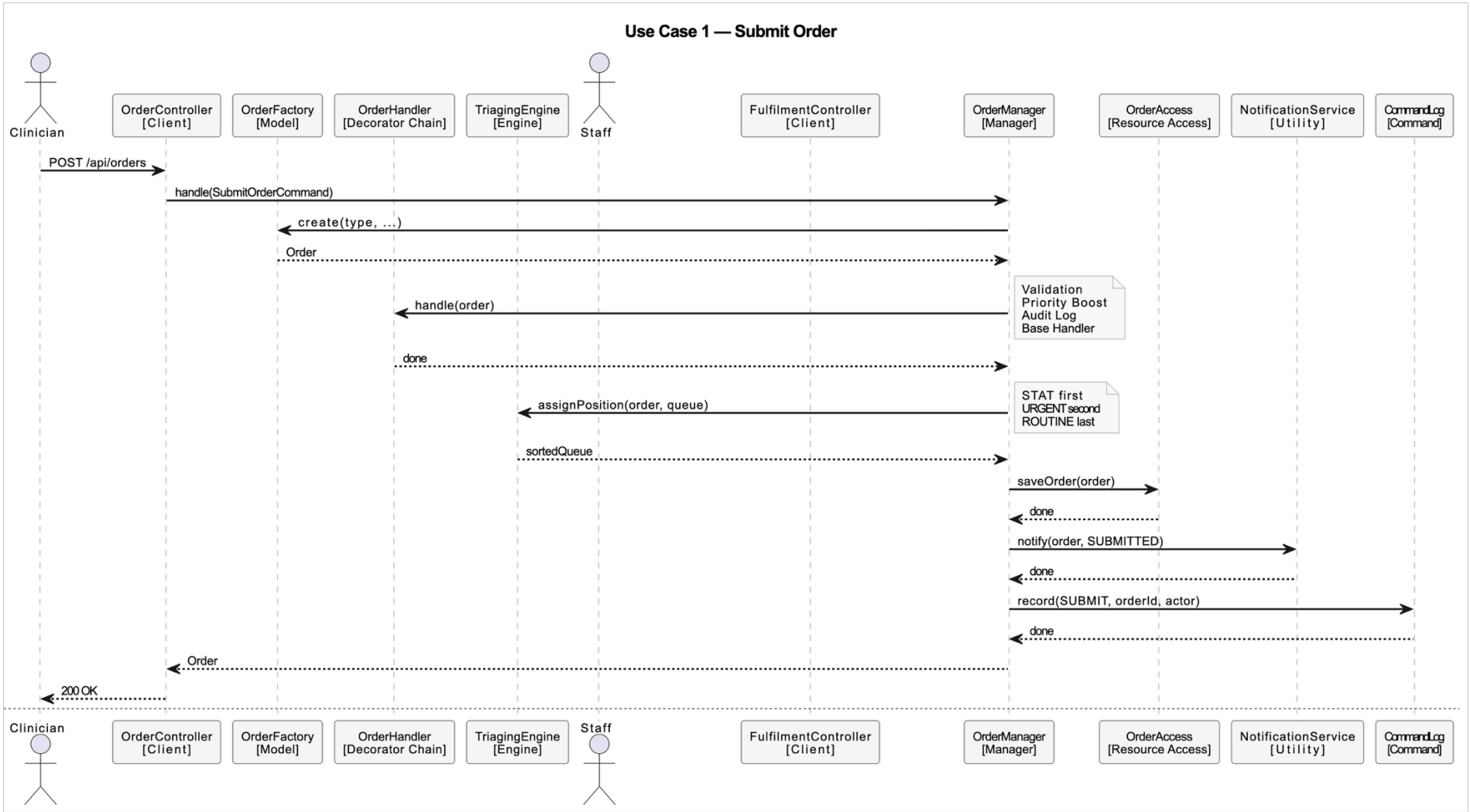
2. Design Patterns — Classes and Justifications

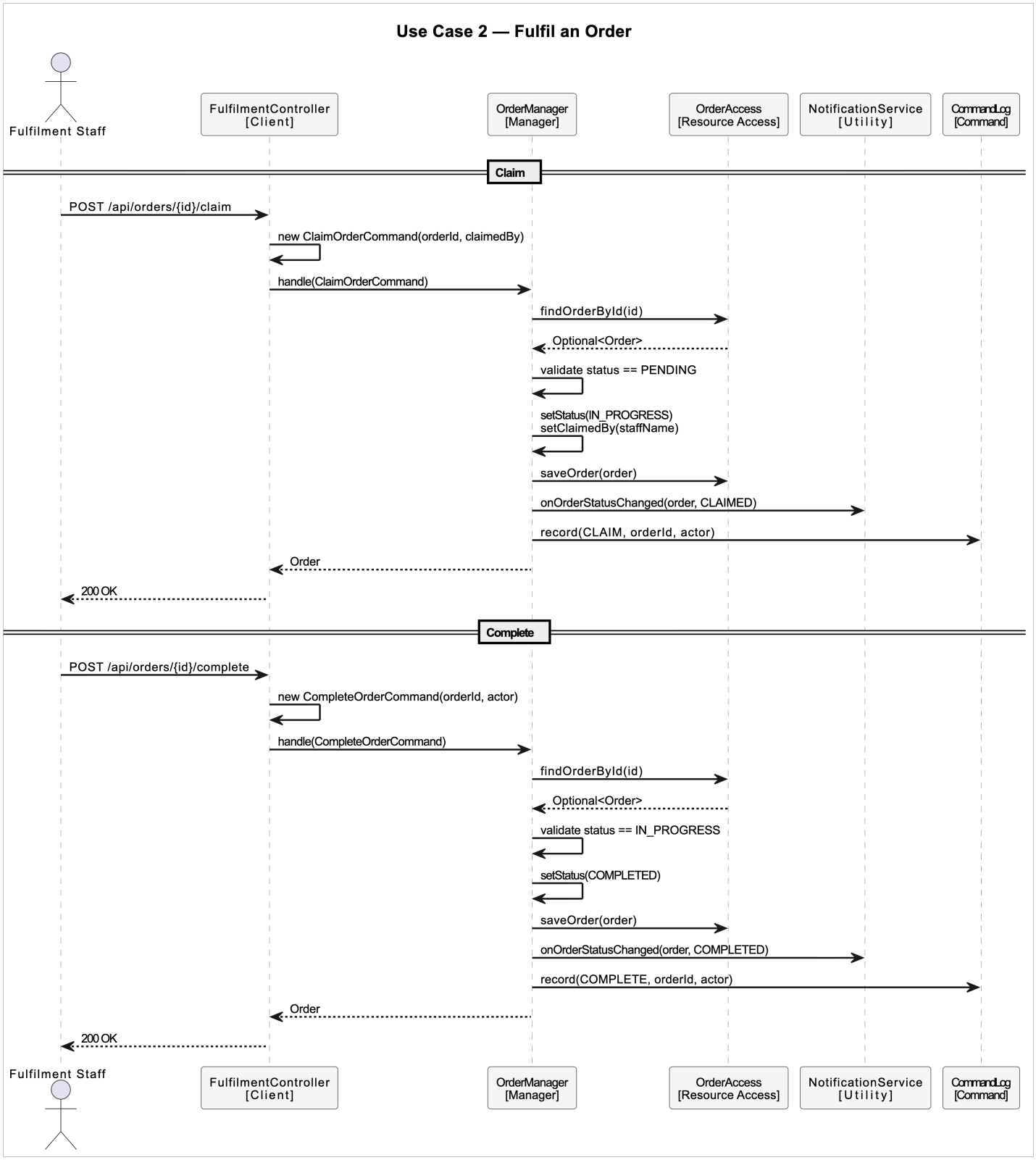
Pattern	Layer	Classes Involved	One-Sentence Justification
Strategy	Layer 2 Engine	TriageStrategy (interface) PriorityFirstStrategy TriagingEngine (context)	Encapsulates the triage algorithm behind an interface so that TriagingEngine can swap algorithms at runtime without modifying OrderManager or any other class.
Observer	Utility	NotificationService (interface) ConsoleNotificationService OrderManager (subject)	Decouples order status changes from notification delivery so that OrderManager calls notify() once and the notification channel can be changed or stacked in Week 2 without touching OrderManager.
Decorator	Layer 2 Engine	OrderHandler (interface) BaseOrderHandler ValidationDecorator PriorityBoostDecorator AuditLoggingDecorator	Stacks order-processing steps transparently around a base handler so that adding a new step (e.g. DuplicateCheckDecorator) in Week 2 requires one new class and one line in AppConfig with no changes to existing handlers.
Factory	Layer 2 Manager	OrderFactory (creator) Order (abstract) LabOrder, MedicationOrder ImagingOrder (products)	Centralises order creation so that OrderManager never calls new LabOrder() directly, meaning a new order type added in Week 2 requires changes in exactly one place.
Command	Layer 2 Manager	Command (interface) SubmitOrderCommand ClaimOrderCommand CompleteOrderCommand CancelOrderCommand CommandLog, CommandLogEntry	Wraps every order action as a self-contained object so that all actions are automatically recorded in the audit log and can be undone in Week 2 without modifying OrderManager's core logic.

3. Call Chain Diagrams — Core Use Cases



Symbol	Meaning	
<	- -	Inheritance
<	. .	Interface implement
o-->	Aggregation	
-->	Dependency	
. . >	Creates	


Use Case 1 — Submit Order
Clinician
OrderController [Client]
OrderFactory [Model]
OrderHandler [Decorator Chain]
TriagingEngine [Engine]
Staff
FulfilmentController [Client]
OrderManager [Manager]
OrderAccess [Resource Access]
NotificationService [Utility]
CommandLog [Command]
POST /api/orders
handle(SubmitOrderCommand)
create(type, ...)
Order
Validation
Priority Boost
Audit Log
Base Handler
handle(order)
done
STAT first
URGENT second
ROUTINE last
assignPosition(order, queue)
sortedQueue
saveOrder(order)
done
notify(order, SUBMITTED)
done
record(SUBMIT, orderId, actor)
done
Order
200 OK


Use Case 2 — Fulfil an Order
Fulfilment Staff
FulfilmentController [Client]
OrderManager [Manager]
OrderAccess [Resource Access]
NotificationService [Utility]
CommandLog [Command]
Claim
POST /api/orders/{id}/claim
new ClaimOrderCommand(orderId, claimedBy)
handle(ClaimOrderCommand)
findOrderById(id)
Optional<Order>
validate status == PENDING
setStatus(IN_PROGRESS)
setClaimedBy(staffName)
saveOrder(order)
onOrderStatusChanged(order, CLAIMED)
record(CLAIM, orderId, actor)
Order
200 OK
Complete
POST /api/orders/{id}/complete
new CompleteOrderCommand(orderId, actor)
handle(CompleteOrderCommand)
findOrderById(id)
Optional<Order>
validate status == IN_PROGRESS
setStatus(COMPLETED)
saveOrder(order)
onOrderStatusChanged(order, COMPLETED)
record(COMPLETE, orderId, actor)
Order
200 OK
Fulfilment Staff
FulfilmentController [Client]
OrderManager [Manager]
OrderAccess [Resource Access]
NotificationService [Utility]
CommandLog [Command]